

IDE

Un *Entorno de Desarrollo Integrado* es una aplicación que reúne herramientas para programar de forma eficiente, como editor de código, depurador, autocompletado, compilador o intérprete, y gestor de proyectos.

Ejemplos: Visual Studio Code, IntelliJ IDEA, Eclipse.

Ciclo de vida del software

Proceso completo que sigue un software desde su concepción hasta su retiro:

1. Análisis de requisitos
 2. Diseño
 3. Desarrollo (implementación)
 4. Pruebas
 5. Implantación o despliegue
 6. Mantenimiento
 7. Retiro
-

Fases en el desarrollo y ejecución del software

Desarrollo

- Recolección y análisis de requisitos
- Diseño de la solución
- Codificación
- Pruebas

Ejecución

- Traducción del código (compilación o interpretación)
- Carga en memoria

- Ejecución del programa
-

Diferencia entre código fuente, código objeto y código ejecutable

- **Código fuente:** texto escrito por el programador en un lenguaje de programación.
 - **Código objeto:** resultado intermedio generado por el compilador, contiene instrucciones en lenguaje máquina pero aún no está listo para ejecutarse solo.
 - **Código ejecutable:** archivo final que el sistema operativo puede ejecutar directamente.
-

Diferencia entre algoritmo, pseudocódigo y código

- **Algoritmo:** conjunto ordenado de pasos lógicos para resolver un problema.
 - **Pseudocódigo:** representación textual del algoritmo con un estilo similar a un lenguaje de programación, pero sin sintaxis formal.
 - **Código:** implementación del algoritmo utilizando un lenguaje de programación real.
-

Diseño estructurado

Metodología de programación orientada a organizar el software de forma clara, modular y fácil de mantener.

Se basa en tres estructuras fundamentales:

- Secuencia (instrucciones en orden)
 - Selección (decisiones: if, switch)
 - Iteración (repetición: for, while)
-

Lenguaje de programación y clasificación

Un lenguaje de programación es un medio formal para escribir instrucciones que la computadora puede interpretar o ejecutar.

Según nivel de abstracción

- Bajo nivel: cercano al hardware
Ejemplo: Ensamblador
- Nivel medio: mezcla entre bajo y alto
Ejemplo: C
- Alto nivel: cercano al lenguaje humano
Ejemplos: Python, Java, JavaScript

Según forma de ejecución

- Compilados
Ejemplos: C, C++
 - Interpretados
Ejemplos: Python, JavaScript
 - Híbridos (compilan a bytecode y ejecutan en máquina virtual)
Ejemplos: Java, C#
-

Diferencia entre compilador e intérprete

- **Compilador:** traduce el código fuente completo a código ejecutable antes de su ejecución.
 - **Intérprete:** traduce y ejecuta las instrucciones línea por línea.
-

Bytecode

Código intermedio generado por algunos lenguajes (como Java y C#) que no corresponde a un procesador físico, sino a una máquina virtual. Facilita portabilidad y ejecución multiplataforma.

POO: clase, objeto y método

- **Clase:** plantilla o modelo que define propiedades y comportamientos.
- **Objeto:** instancia concreta de una clase.
- **Método:** acción o comportamiento asociado a una clase u objeto.

Ejemplo en HTML + JavaScript

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Ejemplo P00</title>
</head>
<body>
  <h2>Ejemplo de Programación Orientada a Objetos</h2>
  <p id="salida"></p>

  <script>
    // Clase
    class Perro {
      constructor(nombre, edad) {
        this.nombre = nombre;
        this.edad = edad;
      }

      // Método
      ladrar() {
        return "Guau. Mi nombre es " + this.nombre + " y tengo " +
this.edad + " años.";
      }
    }

    // Objeto
    const perro1 = new Perro("Rex", 3);

    // Mostrar resultado
    document.getElementById("salida").textContent = perro1.ladrar();
  </script>
</body>
</html>
```

La rama de github:

<https://github.com/hugoexposiito/Tarea-de-ETS/tree/conceptosInicialesExpositoTorres>